

Trabajo Práctico Integrador – Sistema de Gestión "Food Store"

Universidad Tecnológica Nacional - Facultad Regional San Nicolás

Tecnicatura Universitaria en Programación

Programación II

Alejo Garelli

Comisión 6

Fecha de entrega: 22/06/2026

Índice

1. Enlaces del Proyecto (Repositorio y Video)	Pág. 3
2. Marco Teórico.....	Pág. 4
3. Decisiones Técnicas y Arquitectura.....	Pág. 5
4. Dificultades y Soluciones.....	Pág. 7
5. Bibliografía / Webgrafía.....	Pág. 8

(PÁGINA 3: Enlaces Obligatorios)

Enlaces del Proyecto

A continuación, se detallan los enlaces requeridos para la corrección del presente Trabajo Práctico Integrador, cumpliendo con las reglas excluyentes pautadas por la cátedra:

- **Repositorio Público (GitHub):** <https://github.com/alejogarelli/TPI-Programacion2-FoodStore>
- **Video Demostrativo:**
https://drive.google.com/file/d/1B8HvXwiKPU3bbsDoOLy410XKIVFs00nk/view?usp=drive_link

Marco Teórico

Para el desarrollo del sistema "Food Store" se utilizaron los siguientes conceptos y tecnologías, basándonos puramente en la Programación Orientada a Objetos (**POO**).

Lenguaje: Java (versión 21), utilizando Apache NetBeans como IDE principal.

Abstracción: Es el proceso de simplificar la realidad enfocándose únicamente en las características y detalles esenciales de un objeto o sistema, ignorando todo aquello que es irrelevante para resolver el problema. Este principio nos permite manejar la complejidad del software al modelar solo lo que realmente importa.

Herencia: Es un mecanismo fundamental que permite crear nuevas clases (subclases) a partir de clases ya existentes (superclases), heredando automáticamente sus atributos y métodos. Se basa en la relación lógica "es un" (por ejemplo, un Auto *es un* Vehículo) y su principal ventaja es evitar la duplicación promoviendo la reutilización de código y la extensibilidad del sistema.

Polimorfismo e Interfaces: Se implementó la interfaz Calculable para estandarizar el comportamiento de la clase Pedido, asegurando que cuente obligatoriamente con el método calcularTotal().

Colecciones Dinámicas en Memoria: Al no contar con persistencia en base de datos, el almacenamiento se gestionó a través de estructuras ArrayList. Esto nos permitió guardar, buscar, iterar y manipular objetos en memoria RAM de forma dinámica.

Manejo de Excepciones: Se aplicaron bloques try-catch para capturar errores de entrada de datos en la consola (Scanner). Asimismo, se crearon excepciones propias (EntidadNoEncontradaException y DatoInvalidoException) extendiendo de RuntimeException para salvaguardar la integridad de las reglas de negocio sin detener la ejecución del programa.

Decisiones Técnicas y Arquitectura

Modelado UML e Implementación de la Relación 1 a 1 (Regla Excluyente)

Como pauta excluyente de la cátedra, el proyecto debía incluir una relación 1 a 1 real y efectiva tanto en el diseño como en el código. Al analizar la estructura inicial del sistema *Food Store*, noté que las entidades principales (Categoría, Producto, Usuario y Pedido) solo manejaban relaciones de "1 a muchos" o "muchos a 1" (por ejemplo, un usuario puede tener muchos pedidos).

Para resolver esto y cumplir estrictamente con el requisito académico, tomé la decisión de diseño de separar los datos del domicilio del cliente. Para ello, creé una entidad independiente llamada *Direccion*, la cual vinculé de manera unidireccional con el Usuario.

En la práctica, esto funciona de la siguiente manera:

- A nivel de modelo: Un usuario tiene asignada una (1) sola dirección de forma exclusiva, pero la clase *Direccion* "no conoce" a qué usuario pertenece. Esto me permitió garantizar un bajo acoplamiento en el sistema.
- A nivel de código: Dentro de mi paquete *entities*, agregué a la clase *Usuario* un atributo privado que es un objeto (`private Direccion direccion;`). Este valor se lo inyecto de forma obligatoria a través del constructor cada vez que instancio un nuevo usuario.

Para asegurarme de que el código sea fácil de mantener y esté bien cohesionado, decidí organizar el sistema separando las responsabilidades en distintos paquetes lógicos:

- *entities*: Aquí ubiqué las clases principales que representan el modelo de mi negocio. Además, para cumplir con uno de los requisitos excluyentes de la cátedra, armé una relación 1 a 1 estricta entre el Usuario y una nueva clase que creé llamada *Direccion*.
- *services*: En este paquete puse la clase *SistemaService*, que funciona como el verdadero "motor" de la aplicación. Es acá donde viven mis colecciones en memoria (como los *ArrayList*) y donde proceso toda la lógica pesada, como validar el stock, asegurarme de que no haya correos repetidos y hacer las búsquedas.
- *enums e interfaces*: Usé este espacio para agrupar las opciones que son fijas en el sistema (como los roles, estados y formas de pago) y para definir comportamientos estandarizados.

- exceptions: Dediqué este paquete exclusivamente a mis excepciones personalizadas, lo que me permite atajar errores específicos y evitar que el programa se cierre de golpe si algo sale mal.
- ui y principal (Main): Para la interacción, creé la clase AppMenu, encargada de aislar por completo el uso del Scanner y los mensajes por pantalla. Gracias a esto, logré que mi clase Main quede totalmente limpia, limitándose únicamente a instanciar y arrancar el menú de inicio.

Mi decisión técnica sobre la "Baja Lógica" (Soft Delete) En lugar de borrar físicamente los registros de mis listas —lo cual me traería problemas con la integridad del historial en los pedidos—, opté por implementar la baja lógica requerida. Para hacerlo, hice que todas mis entidades hereden de la clase Base un atributo booleano llamado eliminado, que por defecto inicia en false.

Cuando el usuario decide eliminar algo, simplemente cambio ese valor a true. Luego, las funciones encargadas de listar la información dentro de mis servicios filtran automáticamente estos datos, ocultando los registros eliminados para que ya no aparezcan en pantalla.

Dificultades y Soluciones

Durante el desarrollo del proyecto, me encontré con algunos desafíos técnicos que logré resolver de la siguiente manera:

El conflicto con el buffer del *Scanner*: Al principio, cuando alternaba entre pedir números (usando `nextInt()` o `nextLong()`) y pedir texto (con `nextLine()`), notaba que la consola se "saltaba" el ingreso de datos de texto y me terminaba guardando cadenas vacías. Para solucionar esto y lograr una entrada de datos más estable, decidí estandarizar la forma en que uso el *Scanner*: pasé a leer absolutamente todo como texto usando `scanner.nextLine()`. Luego, cuando necesito un valor numérico, simplemente convierto ese texto al tipo correspondiente usando métodos como `Integer.parseInt()` o `Double.parseDouble()`.

Gestión de excepciones inesperadas: Otro problema era que, si el usuario ingresaba letras cuando el sistema esperaba un número, la aplicación se cerraba de golpe, lo cual daba una muy mala experiencia de uso. Para lograr un manejo robusto de errores, envolví toda la interacción de mi capa visual (*AppMenu*) dentro de bloques `try-catch`. De esta manera, capturo cualquier error de tipo `Exception` que pueda surgir, imprimo un mensaje claro en pantalla y devuelvo al usuario al menú principal de forma segura, evitando que el programa se caiga.

Filtrado en las bajas lógicas (*Soft Delete*): Al implementar las relaciones de 1 a N, me di cuenta de un detalle importante: si le daba la baja lógica a un producto, el total del pedido lo seguía contabilizando. Para resolver esto, ajusté la lógica de mi método `calcularTotal()`. Lo que hice fue implementar un ciclo *For-Each* para recorrer los elementos y sumar al total únicamente aquellos `DetallePedido` cuyo estado interno de eliminado fuera falso, respetando así la consistencia de los datos.

Bibliografía / Webgrafía

- **Apuntes Teóricos de la Materia (Programación II):**
 - *U2-A1-Complemento-Practico-Detalles-del-Entorno-y-Errores.pdf*
 - *U4-A3-Complemento-practico-Gestion-de-Estado-Global-con-Static.pdf*
 - *U5-A2-Complemento-Practico-Modelado-de-Relaciones-1-a-1.pdf*
- **Herramientas de Diseño UML:** Diagramas elaborados con la aplicación gratuita de modelado UMLetino.